

When the program is run and the user starts clicking on the balloons, we see output like this in the shell or console window:

```
Small Balloon 2 is going away
Small Balloon 8 is going away
Small Balloon 3 is going away
Small Balloon 7 is going away
Small Balloon 9 is going away
Small Balloon 12 is going away
Small Balloon 11 is going away
Small Balloon 6 is going away
Medium Balloon 14 is going away
Large Balloon 1 is going away
Medium Balloon 10 is going away
Medium Balloon 13 is going away
Medium Balloon 0 is going away
Medium Balloon 4 is going away
Large Balloon 5 is going away
```

When the game is over, the program waits for the user to click on the Start button. When that button is clicked, the balloon manager re-creates the list of `Balloon` objects and resets its instance variables, and the game begins again.

Managing Memory: Slots

As we have discussed, when you instantiate an object, Python must allocate space for the instance variables defined in the class. By default, Python does this using a dictionary with a special name: `__dict__`. To see this in action, you can add this line to the end of the `__init__()` method of any class:

```
print(self.__dict__)
```

A dictionary is an excellent way to represent all the instance variables because it is dynamic—it can grow whenever Python encounters an instance variable that it has not seen before in a class. While I recommend that you initialize all your instance variables in your `__init__()` method, you can in fact define instance variables in any method, and those instance variables will be added when the method is executed for the first time.